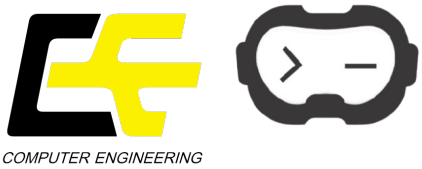


Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikroprosesor

Jordi - 5024241029

2026

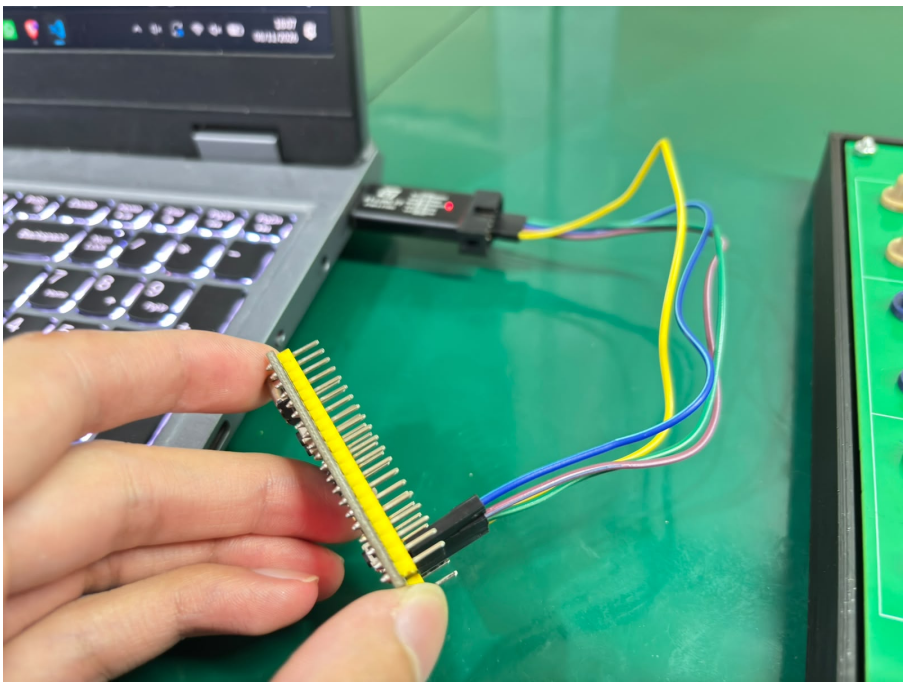
1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Mengisi `platformio.ini` dengan board `blackpill_f401cc`, framework `arduino`, dan `upload_protocol = stlink`.

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menghubungkan ST-Link ke Blackpill: SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3, lalu menyambungkan ST-Link ke komputer melalui USB.



Gambar 1: Gambar sambungan stlink pada Blackpill

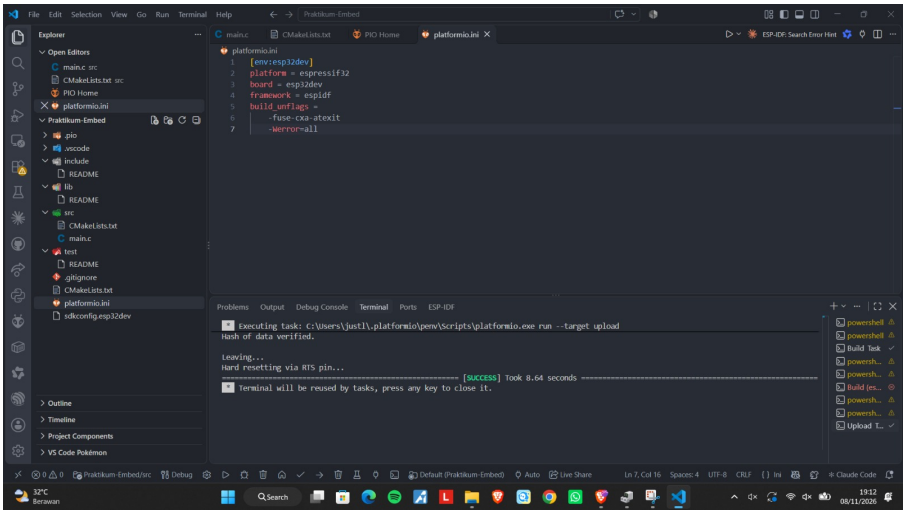
3. Menulis program Blink pada `src/main.cpp` dengan LED onboard di pin PC13 (aktif LOW) dan jeda 500 ms.

4. Menjalankan Build dan Upload, lalu mengamati LED onboard.

Gambar 2: Hasil akhir led pada stm blackpill

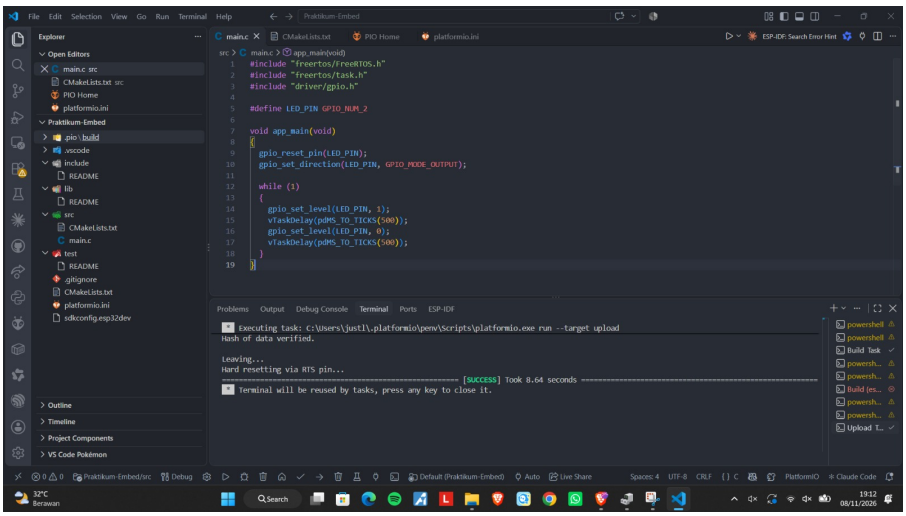
1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Mengganti `platformio.ini` dengan board `esp32dev` dan framework `espidf`.

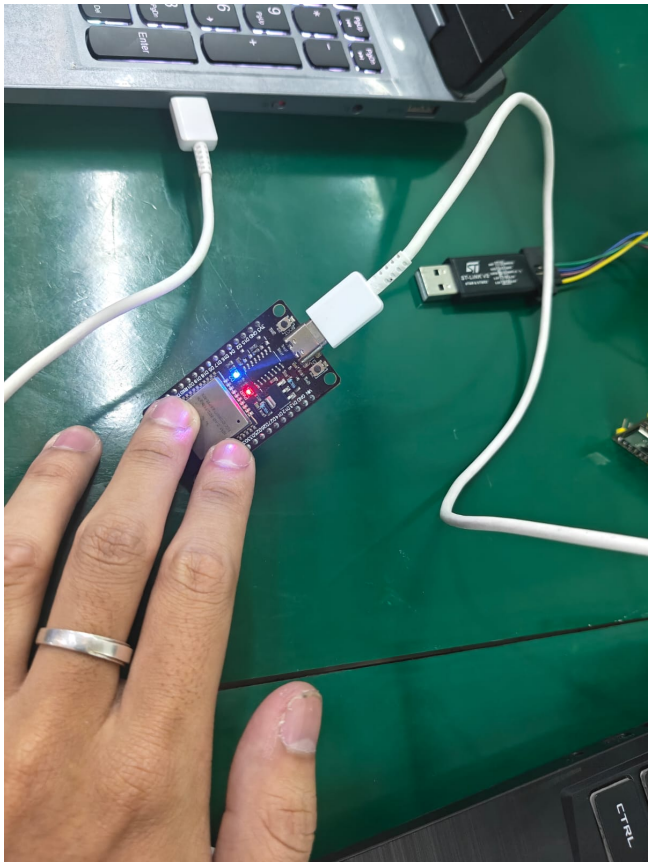


Gambar 3: Pengantian platformio

2. Menyambungkan ESP32 ke komputer melalui USB dan memastikan port serial terdeteksi.
3. Menghapus `src/main.cpp`, membuat `src/main.c`, dan menulis program Blink dengan fungsi `app_main()`, `gpio_set_level()`, dan `vTaskDelay()` pada GPIO 2.
4. Menjalankan Build dan Upload, mengamati LED, lalu membandingkan struktur program ESP-IDF dengan Arduino.



Gambar 4: Gambar kode di vscode untuk percobaan kedua

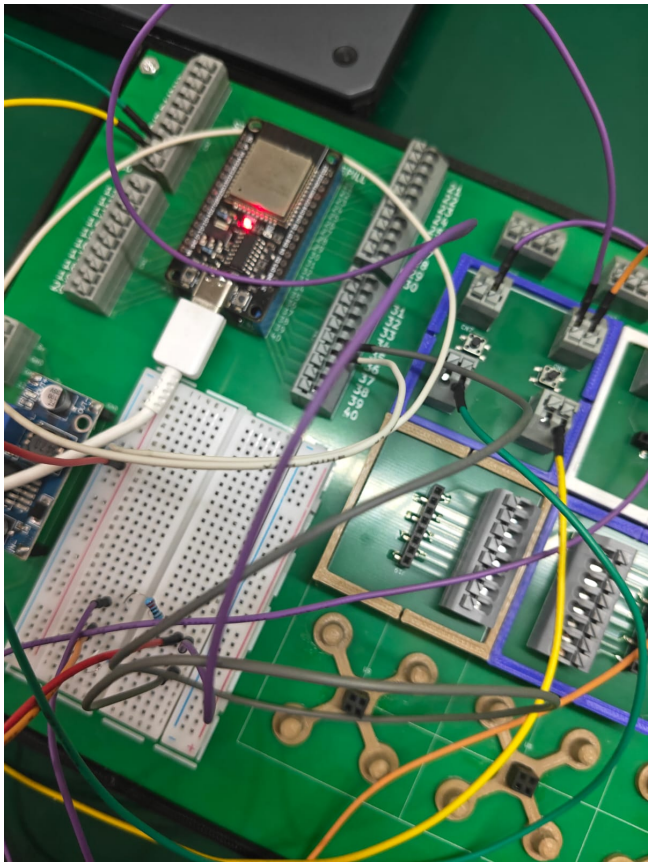


Gambar 5: Hasil akhir led menyala pada esp32

1.3 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal

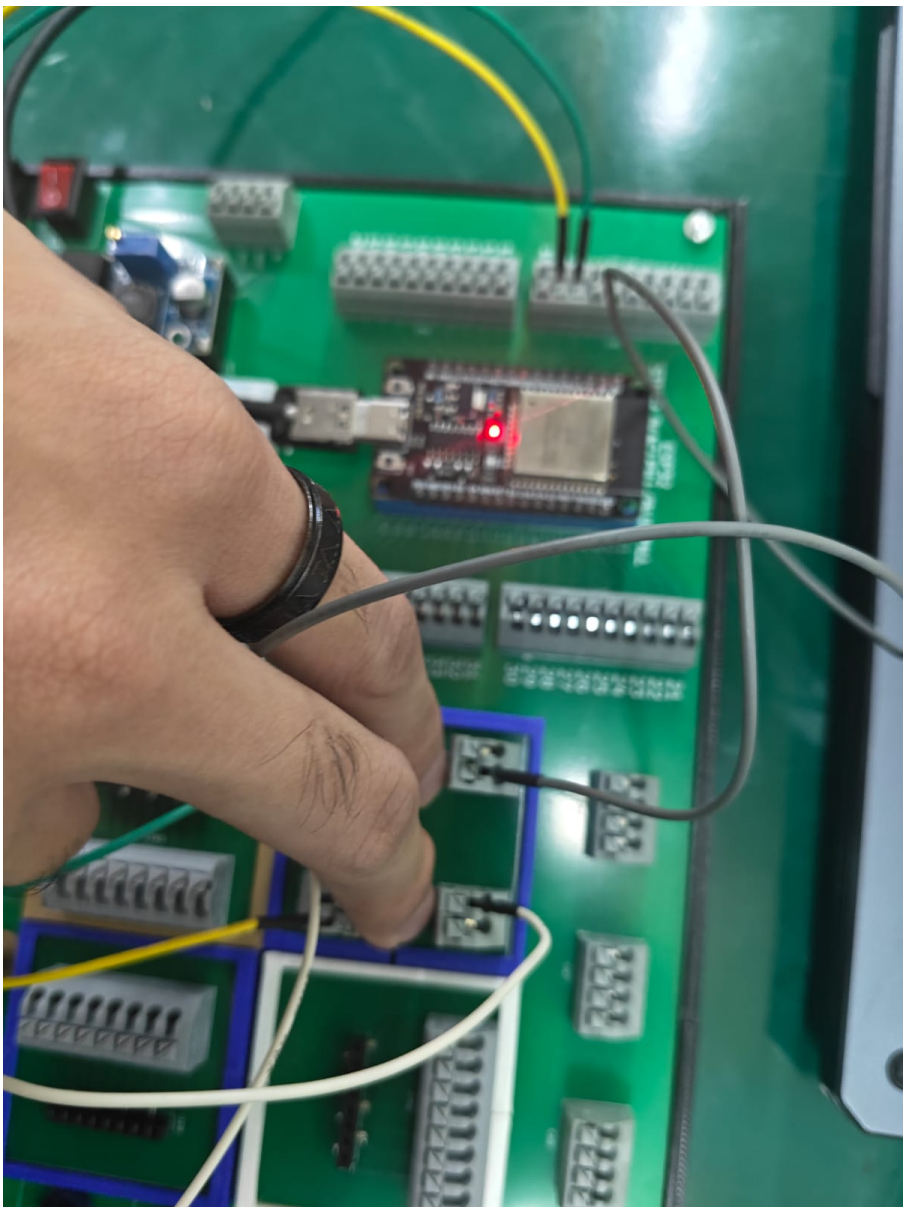
1. Mengembalikan `platformio.ini` ke framework `arduino` dan membuat kembali `src/main.cpp`.
2. **Bagian A (eksternal):** memasang pushbutton 1 pada GPIO 32 dengan resistor 10kΩ ke 3.3V (pull-up) dan kaki lain ke GND, serta pushbutton 2 pada GPIO 33 dengan resistor 10kΩ ke GND (pull-down) dan kaki lain ke 3.3V.
3. Mengunggah Kode Program A (`pinMode(pin, INPUT)`), menekan kedua tombol, dan mencatat

keluaran Serial Monitor.



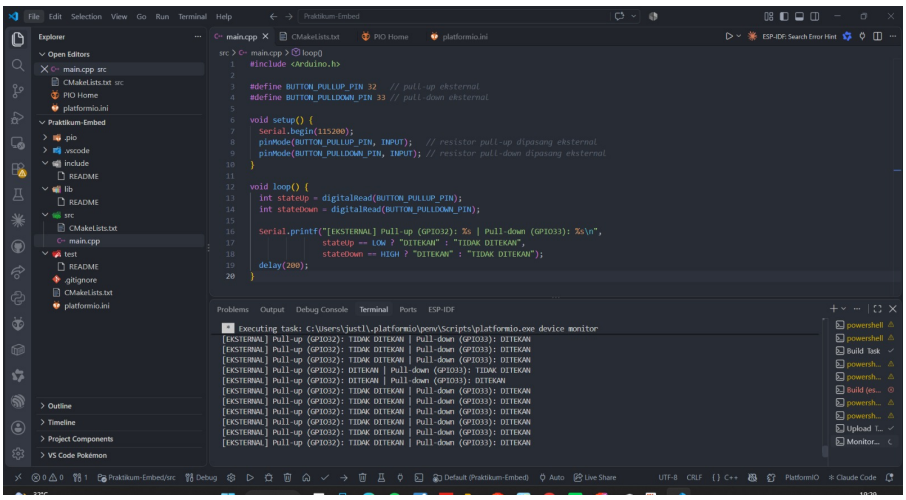
Gambar 6: Rangkaian percobaan ketiga dan kondisi saat ditekan bagian A

4. **Bagian B (internal):** melepas kedua resistor eksternal dengan tombol tetap terpasang pada GPIO yang sama.

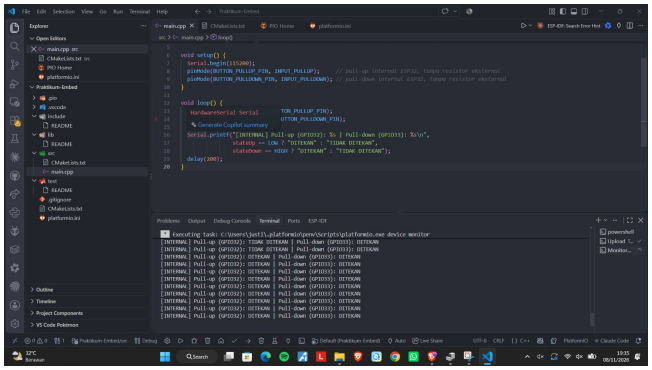


Gambar 7: Rangkaian percobaan ketiga dan kondisi saat ditekan bagian B

5. Mengunggah Kode Program B (`INPUT_PULLUP` dan `INPUT_PULLDOWN`), menekan kedua tombol, dan mencatat keluaran Serial Monitor.
6. Membandingkan hasil Bagian A dan Bagian B.



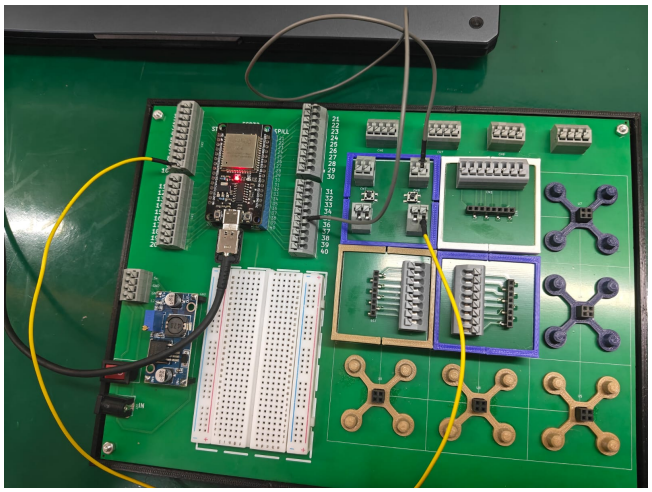
Gambar 8: Gambar hasil pada saat di tekan untuk hasil A



Gambar 9: Gambar hasil pada saat di tekan untuk hasil B

1.4 Percobaan 4: Debouncing pada Input Tombol

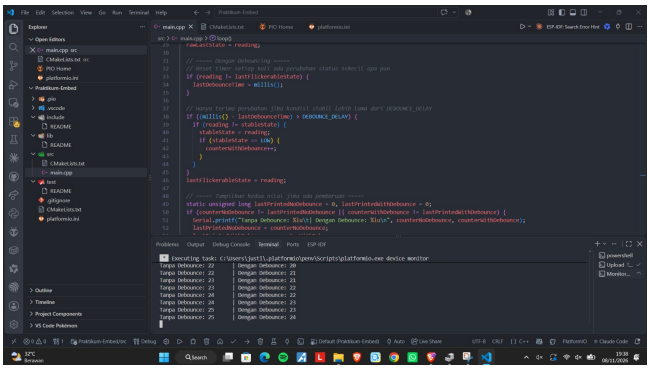
1. Memasang pushbutton pada GPIO 25 dengan kaki lain ke GND dan memakai `INPUT_PULLUP`.



Gambar 10: Gambar rangkaian percobaan 4

2. Mengunggah program dengan dua penghitung, yaitu `counterNoDebounce` dan `counterWithDebounce` (`DEBOUNCE_DELAY` = 50 ms).

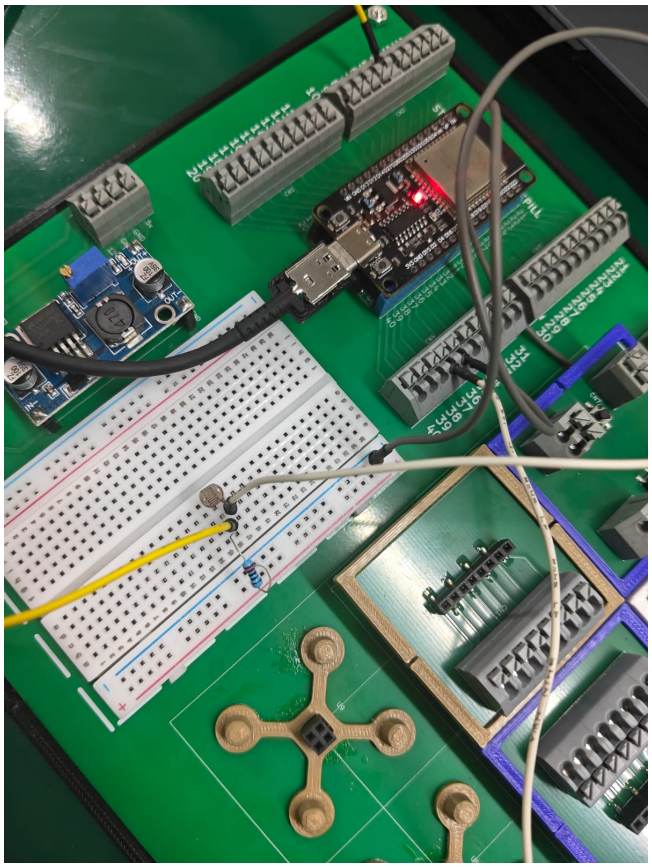
3. Menekan tombol satu kali, baik cepat maupun lambat, lalu membandingkan kedua penghitung pada Serial Monitor.
4. Mengubah `DEBOUNCE_DELAY` menjadi 10 ms, mengunggah ulang, dan menekan tombol kembali.
5. Mengubah `DEBOUNCE_DELAY` menjadi 200 ms, mengunggah ulang, dan menekan tombol dengan cepat berulang kali.
6. Membandingkan hasil untuk nilai 10, 50, dan 200 ms.



Gambar 11: Hasil percobaan keempat

1.5 Percobaan 5: Pembacaan ADC dengan LDR

1. Merangkai LDR sebagai pembagi tegangan: satu kaki LDR ke 3.3V, kaki lain ke GPIO 34 dan ke GND melalui resistor 10kΩ.

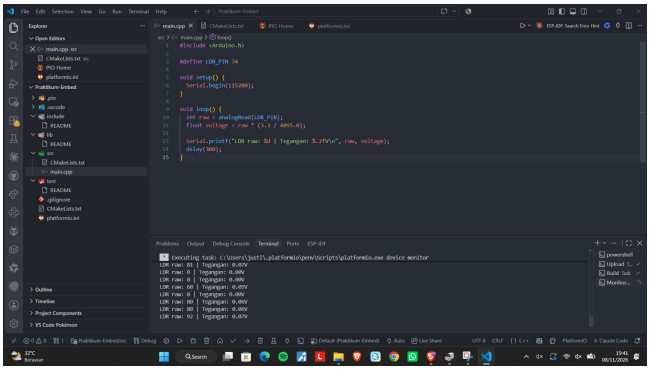


Gambar 12: Rangkaian percobaan kelima

2. Menulis program yang membaca `analogRead()` pada GPIO 34 dan mengonversinya menjadi tegangan dengan $V = \text{raw} \times 3.3/4095$.

3. Menjalankan Build dan Upload, lalu membuka Serial Monitor.

4. Menutup LDR dengan tangan, kemudian menyinarinya dengan lampu, dan mencatat nilai ADC pada setiap kondisi.



Gambar 13: Hasil percobaaan kelima dengan menututp dengan tangan sekaligus jika tersinar

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Blink STM32 Blackpill

Setelah proses upload melalui ST-Link berhasil, LED onboard pada PC13 berkedip dengan periode sekitar satu detik (menyala 500 ms, mati 500 ms). Hasil ini sesuai dengan teori bahwa LED onboard Blackpill bersifat aktif LOW: `digitalWrite(LED_PIN, LOW)` menyalakan LED dan `HIGH` mematikannya.

2.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

Program Blink pada GPIO 2 berjalan dengan baik dan LED berkedip dengan interval 500 ms. Perbedaan utama antara ESP-IDF, Arduino adalah framework tingkat tinggi yang menyederhanakan pemrograman mikrokontroler dengan fungsi siap pakai seperti digitalWrite() dan delay(), sehingga cocok untuk pemula dan prototyping cepat. ESP-IDF adalah framework resmi dari Espressif yang memberi kontrol penuh atas fitur ESP32, seperti FreeRTOS, manajemen daya, konfigurasi Wi-Fi/Bluetooth, dan pengaturan memori secara detail.

2.3 Percobaan 3: Pull-up dan Pull-down

Hasil pengamatan Serial Monitor ditunjukkan pada Tabel 1.

Tabel 1: Status pembacaan GPIO pada konfigurasi eksternal dan internal

Konfigurasi	Kondisi tombol	Teori	Eksternal	Internal
Pull-up (GPIO 32)	Tidak ditekan	HIGH	HIGH	HIGH
Pull-up (GPIO 32)	Ditekan	LOW	LOW	LOW
Pull-down (GPIO 33)	Tidak ditekan	LOW	LOW	LOW
Pull-down (GPIO 33)	Ditekan	HIGH	HIGH	HIGH

Pull-up bernilai HIGH saat tombol tidak ditekan dan LOW saat ditekan, sedangkan pull-down bernilai LOW saat tidak ditekan dan HIGH saat ditekan. Kedua tombol bekerja secara independen, dan hasil konfigurasi eksternal sama dengan konfigurasi internal. Hal ini menunjukkan bahwa resistor pull-up dan pull-down internal ESP32 dapat menggantikan resistor eksternal 10kΩ tanpa mengubah perilaku pembacaan, sekaligus mencegah pin berada pada kondisi *floating*.

2.4 Percobaan 4: Debouncing

Pada penghitung tanpa debounce, satu kali penekanan dapat tercatat lebih dari satu kali karena kontak mekanik tombol memantul (*bouncing*) dan menghasilkan beberapa transisi HIGH–LOW dalam waktu sangat singkat. pada 50 ms, bouncing tersaring dengan baik dan respons masih terasa instan sehingga rentang ini cocok untuk dijadikan deboucing pada pengaplikasiannya. Untuk 10 dan juga 200ms tidak dilakukan pengetesan karena adanya kesalahan dalam melakukan percobaan sehingga 10 dan 200 ms tidak dilakukan.

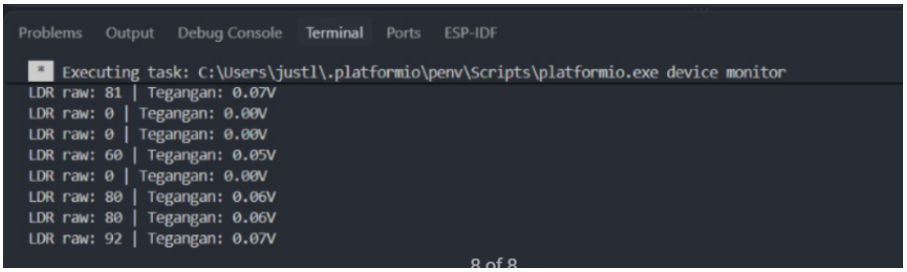
2.5 Percobaan 5: ADC dan LDR

Hasil pembacaan ADC ditunjukkan pada Tabel 2.

Tabel 2: Hasil pembacaan ADC LDR pada GPIO 34

Kondisi cahaya	**Nilai ADC (0–4095)**	**Tegangan (V)**
LDR ditutup (tanpa cahaya)	**0**	**0**
Kondisi ruangan normal	**+-84**	**0,05**

Saat LDR ditutup, resistansinya besar sehingga tegangan pada GPIO 34 rendah dan nilai ADC kecil. Pada kondisi ruangan normal, resistansi LDR turun karena menerima cahaya sehingga tegangan dan nilai ADC lebih besar. Hal ini sesuai dengan sifat LDR, yaitu resistansinya menurun seiring bertambahnya intensitas cahaya. Dari percobaan didapatkan kedua kondisi dapat berjalan sesuai aturan dengan saat ditutup kondisinya menjadi 0 dan saat kondisi ruangan normal menjadi 80-92 sesuai dengan kondisi cahaya ruangan. Untuk kondisi saat di berikan cahaya lewat senter tidak dilakukan karena ada kesalahan saat melakukan praktikum, namun dari hasil dapat di lakukan bahwa dalam percobaan ini berhasil karena sifar dari LDR sudah sesuai teori dan dapat mengolah kondisi cahaya-nya



Gambar 14: Kondisi Serial monitor

3 Hasil Tugas Modul

3.1 Rancangan dan Source Code

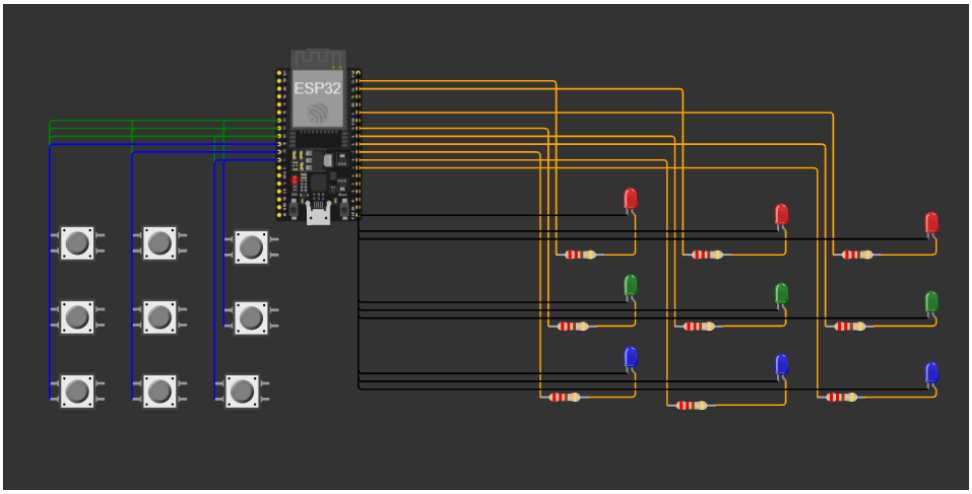
Tabel 3: Pemetaan pin matrix pushbutton dan matrix LED

Fungsi	Baris/Kolom 1	Baris/Kolom 2	Baris/Kolom 3
Baris tombol (output)	GPIO 32	GPIO 33	GPIO 25
Kolom tombol (input pull-up)	GPIO 26	GPIO 27	GPIO 14
LED baris 1	GPIO 23	GPIO 22	GPIO 21
LED baris 2	GPIO 19	GPIO 18	GPIO 5
LED baris 3	GPIO 17	GPIO 16	GPIO 4

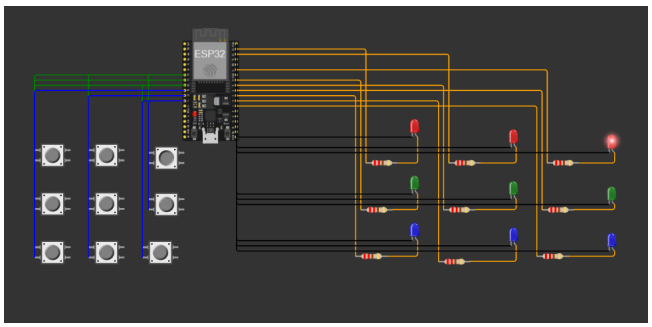
```
const uint8_t ROW_PINS[3] = {32, 33, 25};
const uint8_t COL_PINS[3] = {26, 27, 14};

const uint8_t LED_PINS[3][3] = {
  {23, 22, 21},
  {19, 18,  5},
  {17, 16,  4}
};

const unsigned long DEBOUNCE_MS = 30;
const bool TOGGLE_MODE = false;

bool lastRaw[3][3];
bool stableState[3][3];
bool ledState[3][3];
unsigned long lastChange[3][3];

bool readRaw(int r, int c) {
  for (int i = 0; i < 3; i++) digitalWrite(ROW_PINS[i], i == r ? LOW : HIGH);
  delayMicroseconds(50);
  return digitalRead(COL_PINS[c]) == LOW;
}

void setup() {
  Serial.begin(115200);

  for (int i = 0; i < 3; i++) {
    pinMode(ROW_PINS[i], OUTPUT);
    digitalWrite(ROW_PINS[i], HIGH);
    pinMode(COL_PINS[i], INPUT_PULLUP);
  }

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      pinMode(LED_PINS[r][c], OUTPUT);
      digitalWrite(LED_PINS[r][c], LOW);
      lastRaw[r][c] = false;
      stableState[r][c] = false;
      ledState[r][c] = false;
      lastChange[r][c] = 0;
```

```
    }
  }
  Serial.println("Matrix 3x3 siap (debounce aktif)");
}

void loop() {
  unsigned long now = millis();

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      bool raw = readRaw(r, c);
      if (raw != lastRaw[r][c]) {
        lastChange[r][c] = now;
        lastRaw[r][c] = raw;
      }
      if ((now - lastChange[r][c]) >= DEBOUNCE_MS && raw != stableState[r][c]) {
        stableState[r][c] = raw;

        if (TOGGLE_MODE) {
          if (stableState[r][c]) ledState[r][c] = !ledState[r][c];
        } else {
          ledState[r][c] = stableState[r][c];
        }

        digitalWrite(LED_PINS[r][c], ledState[r][c] ? HIGH : LOW);

        Serial.printf("Tombol (%d,%d) %s -> LED (%d,%d) %s\n",
                      r + 1, c + 1, stableState[r][c] ? "DITEKAN" : "DILEPAS",
                      r + 1, c + 1, ledState[r][c] ? "ON" : "OFF");
      }
    }
  }
}
```

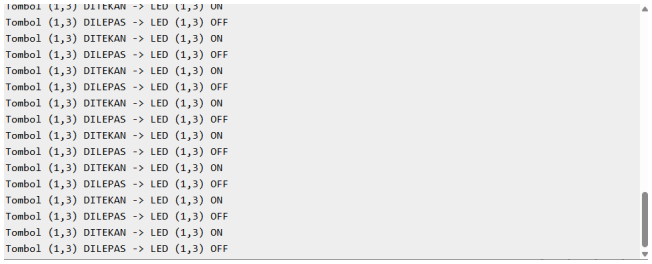
3.2 Hasil Simulasi



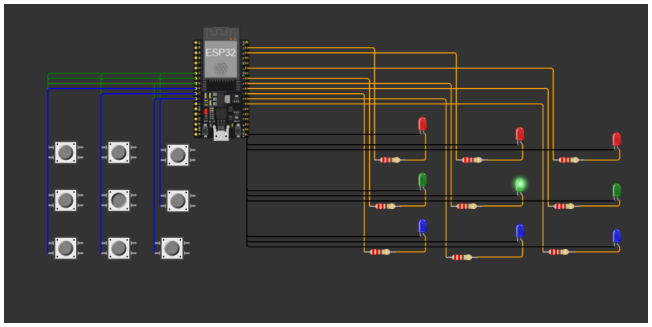
Gambar 15: Gambar rangkaian menggunakan esp32 dan juga matrix tombol dan led



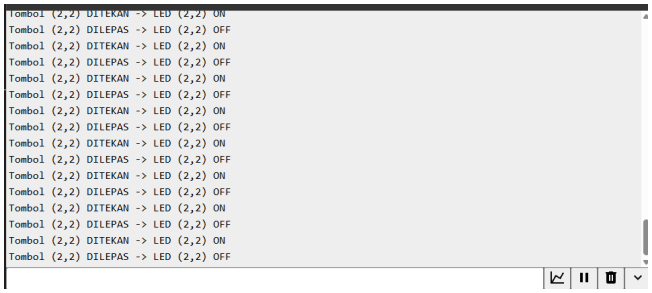
Gambar 16: Percobaan pada a13



Gambar 17: Gambar kondisi pada serial monitor untuk a13



Gambar 18: Percobaan pada a22



Gambar 19: Gambar kondisi pada serial monitor untuk a22

3.3 Jawaban Pertanyaan Analisis

1. Mengapa debouncing tetap diperlukan walaupun hanya satu tombol ditekan pada satu waktu? Bouncing disebabkan oleh sifat fisik kontak mekanik, bukan oleh jumlah tombol yang ditekan. Saat kontak logam bersentuhan atau terlepas, getaran membuat kontak terputus dan tersambung berulang kali dalam beberapa milidetik. Tanpa debouncing, satu penekanan dapat terbaca sebagai beberapa transisi HIGH–LOW sehingga LED ter-toggle berkali-kali dan keadaan akhirnya acak (bisa tetap pada kondisi semula). Bouncing juga terjadi saat tombol dilepas.

2. Konfigurasi pull-up atau pull-down pada kolom tombol. Rancangan ini memakai pull-up internal (`INPUT_PULLUP`) pada pin kolom (GPIO 19, 21, 22). Saat tombol tidak ditekan, kolom ditarik ke 3.3V sehingga terbaca **HIGH**. Saat sebuah baris diberi LOW dan tombol pada baris-kolom tersebut ditekan, kolom terhubung ke baris yang LOW sehingga terbaca **LOW**. Pin tidak boleh dibiarkan *floating* karena pin input tanpa referensi tegangan memiliki impedansi sangat tinggi sehingga nilainya ditentukan noise dan induksi sekitar. Pembacaan dapat berubah acak antara HIGH dan LOW tanpa ada tombol yang ditekan, sehingga LED menyala atau mati sendiri (*false trigger*). Pull-up memberi level HIGH yang pasti pada kondisi idle.

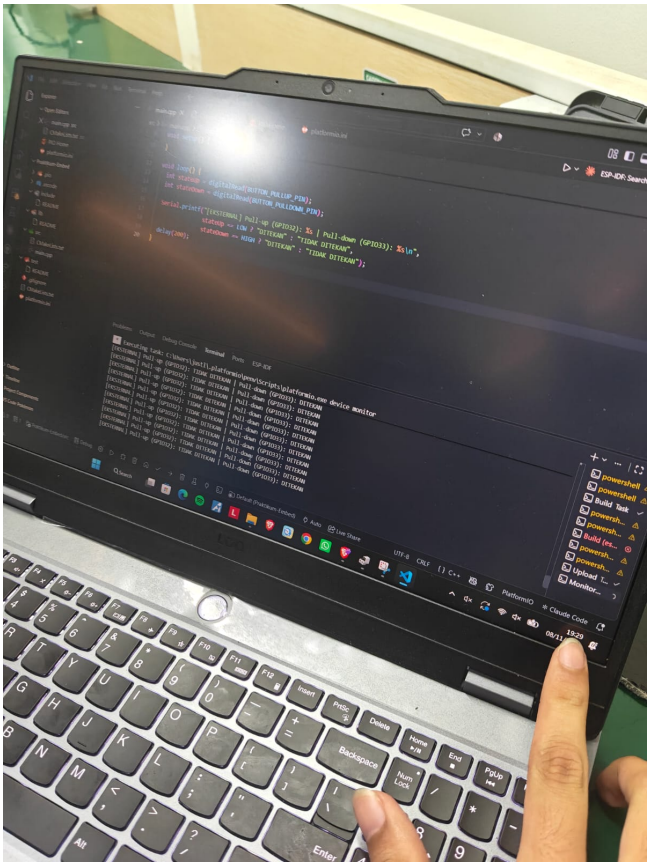
3. Perbedaan digital input dan analog input (ADC). Digital input hanya menghasilkan dua kondisi, HIGH atau LOW, karena tegangan pin dibandingkan dengan ambang (*threshold*) tertentu dan hasilnya hanya 1 bit. Cocok untuk tombol yang hanya berstatus ditekan atau tidak. Analog input melalui ADC mengukur besar tegangan pada pin yang berubah kontinu dan mengubahnya menjadi bilangan digital. ADC 12-bit pada ESP32 menghasilkan $2^{12} = 4096$ level (0–4095) untuk rentang sekitar 0–3.3V, sehingga resolusinya sekitar 0.8 mV per langkah.

4 Kesimpulan

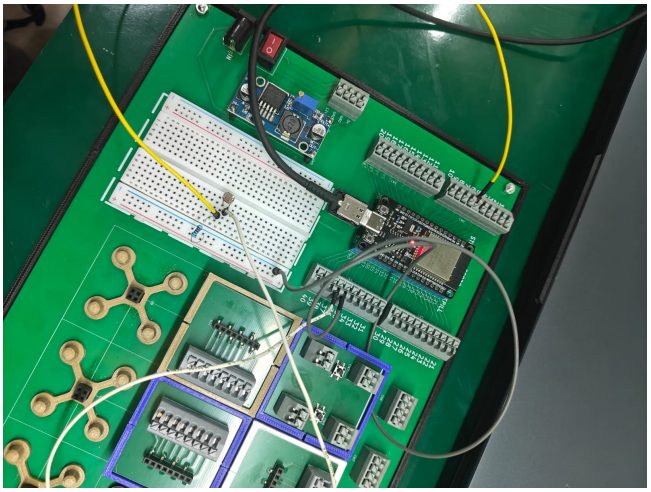
Pada Percobaan 1 dan 2, program Blink berhasil dijalankan pada STM32 Blackpill dengan framework Arduino dan pada ESP32 dengan framework ESP-IDF, sehingga terlihat bahwa Arduino lebih sederhana sedangkan ESP-IDF lebih native dengan `app_main()` dan FreeRTOS. Pada Percobaan 3, konfigurasi pull-up dan pull-down eksternal maupun internal memberikan pembacaan tombol yang sama, sehingga resistor internal ESP32 dapat menggantikan resistor eksternal dan mencegah pin berada pada kondisi *floating*. Pada Percobaan 4, debouncing hanya diuji dengan `DEBOUNCE_DELAY` 50 ms, dan hasilnya menunjukkan penghitung dengan debounce lebih sesuai dengan jumlah penekanan dibandingkan penghitung tanpa debounce, sedangkan pengaruh nilai delay lain belum teramati. Pada Percobaan 5, nilai ADC LDR lebih kecil saat ditutup dan lebih besar pada kondisi ruangan normal, sedangkan kondisi terkena cahaya senter tidak diuji sehingga respons pada cahaya yang lebih terang belum diketahui. Dari kelima percobaan didapatkan pemahaman tentang penggunaan GPIO sebagai digital output, digital input, dan analog input. Pada Tugas Modul, pemahaman tersebut diterapkan pada matrix pushbutton 3×3 pengendali matrix LED 3×3 dengan scanning dan debouncing per tombol sehingga LED hanya berubah setelah pembacaan tombol stabil.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 20: Pengerjaan kode dan upload



Gambar 21: Pengerjaan rangkaian